

PhysHazeDiffusion 方法总结

基于雾密度扩散与大气光物理渲染的真实雾图生成

PhysHazeDiffusion Project

2026 年 6 月 8 日

摘要

本文档总结当前工程中的 PhysHazeDiffusion / PhysHazeGen 分支。该方法不让扩散模型直接生成自由 RGB 雾图或雾残差，而是让网络生成低维、可解释的雾密度场 $d(\mathbf{x})$ ，同时由大气光头或真实雾域大气光库给出全局大气光 $\mathbf{A}$ 。最终通过物理成像模型

$$\mathbf{I}(\mathbf{x}) = \mathbf{J}(\mathbf{x})(1 - d(\mathbf{x})) + \mathbf{A}d(\mathbf{x})$$

将清晰图像 $\mathbf{J}$ 合成为雾图 $\mathbf{I}$ 。这种设计把几何结构、颜色控制和图像生成解耦，减少 RGB residual 方法中常见的偏色、纹理污染和不受控域迁移问题。

1 符号约定

表 1: 核心符号

符号	含义
$\mathbf{J} \in [0, 1]^{3 \times H \times W}$	清晰图像 clean image
$\mathbf{I} \in [0, 1]^{3 \times H \times W}$	合成或真实雾图 hazy image
$t(\mathbf{x}) \in [0, 1]$	透射率 transmission
$d(\mathbf{x}) \in [0, 1]$	雾密度 haze density, 满足 $d = 1 - t$
$\mathbf{A} \in [0, 1]^3$	全局大气光 atmospheric light
$\mathbf{C} \in [-1, 1]^{3 \times H \times W}$	density carrier, 给 VAE / diffusion 使用的三通道载体
z_0	VAE 编码后的 density carrier latent
$\mathcal{E}_{\text{VAE}}, \mathcal{D}_{\text{VAE}}$	VAE 编码器与解码器
ϵ_θ 或 f_θ	条件扩散 / ControlNet 去噪网络

2 物理成雾模型

2.1 经典大气散射模型

单幅图像去雾与成雾通常使用大气散射模型：

$$\mathbf{I}(\mathbf{x}) = \mathbf{J}(\mathbf{x})t(\mathbf{x}) + \mathbf{A}(1 - t(\mathbf{x})), \quad (1)$$

其中 $\mathbf{x}$ 表示像素位置, $\mathbf{J}$ 是无雾清晰图像, $\mathbf{I}$ 是观测雾图, t 是透射率, $\mathbf{A}$ 是全局大气光。若场景深度为 $D(\mathbf{x})$, 介质散射系数为 β , 常见透射率形式为:

$$t(\mathbf{x}) = \exp(-\beta D(\mathbf{x})). \quad (2)$$

深度越大或散射系数越强, t 越小, 远处物体越容易被大气光覆盖。

2.2 从 transmission 到 density

本工程将模型改写为雾密度形式:

$$d(\mathbf{x}) = 1 - t(\mathbf{x}). \quad (3)$$

代入式 (1) 得到:

$$\mathbf{I}(\mathbf{x}) = \mathbf{J}(\mathbf{x})(1 - d(\mathbf{x})) + \mathbf{A}d(\mathbf{x}). \quad (4)$$

因此, $d(\mathbf{x})$ 可以直接理解为当前像素从清晰图像颜色混合到大气光颜色的权重:

- $d(\mathbf{x}) = 0$: 完全保留清晰图像, $\mathbf{I}(\mathbf{x}) = \mathbf{J}(\mathbf{x})$;
- $d(\mathbf{x}) = 1$: 完全由大气光决定, $\mathbf{I}(\mathbf{x}) = \mathbf{A}$;
- $0 < d(\mathbf{x}) < 1$: 清晰图像和大气光线性混合。

2.3 与代码实现的对应关系

物理渲染器在 `phys_haze_utils.py` 中实现为:

```
def compose_physical_haze(clean, density, airlight):
    a = airlight.view(-1, 3, 1, 1).to(device=clean.device, dtype=clean.dtype)
    density = density.clamp(0.0, 1.0)
    return (clean * (1.0 - density) + a * density).clamp(0.0, 1.0)
```

这正是式 (4) 的逐像素实现。

3 Density Carrier 表示

3.1 为什么需要 carrier

Stable Diffusion 的 VAE 和 UNet 原本面向三通道 RGB 图像。雾密度 d 是单通道标量场, 若直接作为目标会与原始 VAE 输入格式不一致。因此工程中把单通道 density 扩展为三通道 carrier:

$$\mathbf{C} = 2 \cdot \text{repeat}_3(d) - 1, \quad (5)$$

其中 $\mathbf{C} \in [-1, 1]^{3 \times H \times W}$ 。训练时使用 VAE 编码:

$$z_0 = \mathcal{E}_{\text{VAE}}(\mathbf{C}). \quad (6)$$

扩散模型学习的是 z_0 的条件分布, 而不是 RGB 雾图或 RGB residual。

3.2 从 carrier 解回 density

推理时扩散模型采样得到 latent $\hat{z}_0$ ，经 VAE 解码得到 carrier：

$$\hat{C} = \mathcal{D}_{\text{VAE}}(\hat{z}_0). \quad (7)$$

再把三通道 carrier 平均后映射回 $[0, 1]$ ：

$$\hat{d} = \text{clamp} \left(\frac{1}{2} \left(\frac{1}{3} \sum_{c=1}^3 \hat{C}_c + 1 \right), 0, 1 \right). \quad (8)$$

代码对应：

```
def density_to_carrier(density):
    return density.expand(-1, 3, -1, -1) * 2.0 - 1.0

def carrier_to_density(carrier_m11):
    return ((carrier_m11.mean(dim=1, keepdim=True) + 1.0) * 0.5).clamp(0.0, 1.0)
```

4 大气光 A 的建模

4.1 AirlightHead

工程中使用一个轻量 CNN 从 clean image 预测低维大气光：

$$\hat{A} = 0.55 + 0.45 \cdot \sigma(g_\phi(J)). \quad (9)$$

这里 g_ϕ 是 AirlightHead，输出三维 RGB 向量。该参数化把 A 限制在 $[0.55, 1.0]$ ，避免大气光变成任意 RGB 颜色，使它保持“亮雾幕”的物理含义。

4.2 从真实雾图构建 A-bank

真实域适配阶段还会从真实雾图中估计大气光库。对雾图 I ，代码选择亮度最高的 top- k 像素并求 RGB 均值：

$$A^* = \frac{1}{|\Omega_k|} \sum_{x \in \Omega_k} I(x), \quad \Omega_k = \text{TopK}_x(\text{mean}_c I_c(x)). \quad (10)$$

之后在 Stage2 中从 bank 中随机采样 A ，使生成雾图的大气光分布靠近真实雾域。

5 网络框架

5.1 整体设计

整体框架可以分为四个模块：

1. **条件编码模块**：输入 clean image、可选 depth condition 和文本 prompt，构造 ControlNet 条件；
2. **Density Diffusion**：条件扩散模型生成 density carrier latent；
3. **Airlight 模块**：AirlightHead 预测 A ，或从真实雾域 A-bank 采样 A ；
4. **Physical Renderer**：使用式 (4) 将 J 、 d 、 A 合成为雾图。

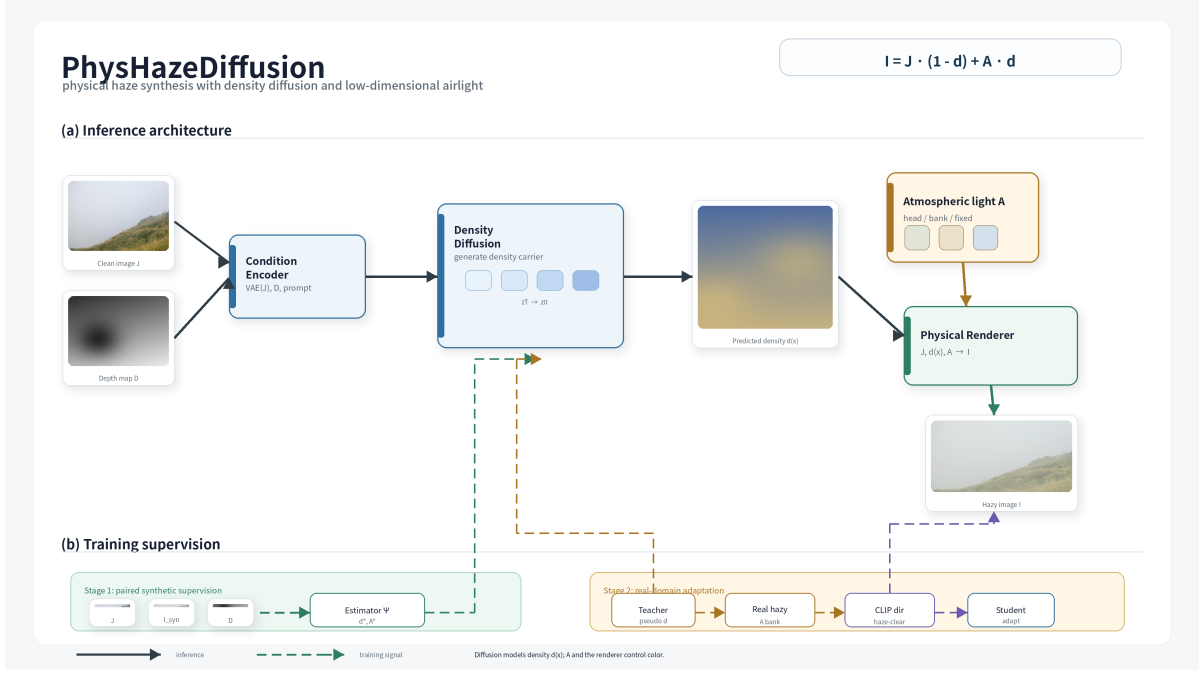


图 1: PhysHazeDiffusion 网络框架示意图。扩散模型只负责 density，颜色由大气光和物理渲染器控制。

5.2 ControlNet 条件

配置文件 `configs/train/phys_stage.yaml` 中，ControlNet 的 hint 通道数为 5：

```
controlnet_cfg:
  in_channels: 4
  hint_channels: 5
```

其含义是：基础 latent 通道仍为 4，条件 hint 由 clean 图像相关条件和可选深度条件组成。深度只作为几何 cue 使用，帮助 density 形成合理的远近结构，而不直接改变 RGB 颜色。

6 Stage1: 合成配对监督

6.1 训练目标构造

Stage1 使用合成配对数据 (J, I) 。首先从配对图像估计大气光 A^* ，再根据物理模型反解 density target。由式 (4) 可得：

$$I - J = d(A - J). \quad (11)$$

逐通道近似反解：

$$d_c^*(x) = \frac{I_c(x) - J_c(x)}{A_c^* - J_c(x) + \varepsilon}, \quad (12)$$

然后对 RGB 通道求均值，并做低通平滑：

$$d^*(x) = \text{Blur} \left(\frac{1}{3} \sum_{c=1}^3 \text{clamp}(d_c^*(x), 0, 1) \right). \quad (13)$$

代码中分母下限为 0.08，并使用 Gaussian blur，使 density target 更偏向低频、保守的雾浓度分布。

6.2 Stage1 损失函数

将 d^* 转为 carrier 并编码:

$$z_0^* = \mathcal{E}_{\text{VAE}}(2 \cdot \text{repeat}_3(d^*) - 1). \quad (14)$$

扩散训练使用标准 DDPM 损失。若模型采用噪声预测参数化, 可写为:

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{t, \epsilon} \left[\|\epsilon - \epsilon_\theta(z_t, t, \mathbf{c})\|_2^2 \right], \quad z_t = \sqrt{\bar{\alpha}_t} z_0^* + \sqrt{1 - \bar{\alpha}_t} \epsilon. \quad (15)$$

同时训练 AirlightHead:

$$\mathcal{L}_A = \text{SmoothL1}(\hat{\mathbf{A}}, \mathbf{A}^*). \quad (16)$$

Stage1 总损失为:

$$\mathcal{L}_{\text{stage1}} = \mathcal{L}_{\text{diff}} + \lambda_A \mathcal{L}_A. \quad (17)$$

7 Stage2: 真实雾域适配

7.1 动机

Stage1 的 density 学自合成配对数据, 雾结构较稳定, 但合成数据的大气光分布、雾色和真实雾图仍可能有域差异。Stage2 的目标是在不引入自由 RGB pseudo target 的前提下, 让模型向真实雾域靠近。

7.2 Teacher-Student pseudo density

Stage2 从 Stage1 checkpoint 初始化 teacher 和 student。Teacher 冻结, 只采样 pseudo density latent:

$$\tilde{z}_0 = \text{Sample}_{\theta_T}(\mathbf{J}, D, p), \quad (18)$$

其中 D 是可选深度条件, p 是 haze prompt。Student 通过扩散损失学习 $\tilde{z}_0$:

$$\mathcal{L}_{\text{pseudo}} = \mathbb{E}_{t, \epsilon} \left[\|\epsilon - \epsilon_{\theta_S}(\tilde{z}_t, t, \mathbf{c})\|_2^2 \right]. \quad (19)$$

注意 teacher 不提供 RGB 雾图, 只提供 density latent, 因此不会把 teacher 可能存在的 RGB 偏色直接传给 student。

7.3 CLIP 方向约束

传统 prototype loss 容易把生成图整体拉向“雾图中心”, 造成曝光和颜色漂移。本方法使用方向约束: 比较图像从 clean 到 pred-hazy 的 CLIP 变化方向, 是否与文本从 clear 到 haze 的方向一致。

文本方向:

$$\Delta_{\text{text}} = \text{norm}(F_{\text{text}}(p_{\text{haze}}) - F_{\text{text}}(p_{\text{clear}})). \quad (20)$$

图像方向:

$$\Delta_{\text{img}} = \text{norm}(F_{\text{img}}(\hat{\mathbf{I}}) - F_{\text{img}}(\mathbf{J})). \quad (21)$$

方向损失：

$$\mathcal{L}_{\text{clip}} = 1 - \langle \Delta_{\text{img}}, \Delta_{\text{text}} \rangle. \quad (22)$$

其中 $\hat{\mathbf{I}}$ 由物理 renderer 得到：

$$\hat{\mathbf{I}} = \mathbf{J}(1 - \hat{d}) + \hat{\mathbf{A}}\hat{d}. \quad (23)$$

7.4 真实大气光分布约束

Stage2 构建真实雾图 A-bank，并采样 $\mathbf{A}_{\text{real}}$ 。AirlightHead 的预测 $\hat{\mathbf{A}}$ 通过 SmoothL1 靠近真实雾域大气光分布：

$$\mathcal{L}_{A,\text{real}} = \text{SmoothL1}(\hat{\mathbf{A}}, \mathbf{A}_{\text{real}}). \quad (24)$$

Stage2 总损失为：

$$\mathcal{L}_{\text{stage2}} = \mathcal{L}_{\text{pseudo}} + \lambda_{\text{clip}}\mathcal{L}_{\text{clip}} + \lambda_{A,2}\mathcal{L}_{A,\text{real}}. \quad (25)$$

8 推理流程

给定一张清晰图 $\mathbf{J}$ ，推理流程如下：

1. 根据 clean image、prompt 和可选 depth 构造条件 $\mathbf{c}$ ；
2. 使用扩散采样器得到 density carrier latent $\hat{z}_0$ ；
3. VAE 解码 $\hat{z}_0$ 得到 carrier，并用式 (8) 得到 $\hat{d}$ ；
4. 选择大气光来源：AirlightHead、真实 A-bank 或固定 $\mathbf{A}$ ；
5. 使用物理 renderer：

$$\hat{\mathbf{I}} = \mathbf{J}(1 - \hat{d}) + \mathbf{A}\hat{d}.$$

核心代码对应 inference_phys_hazegen.py：

```
z = sampler.sample(...)
density = carrier_to_density(model.vae_decode(z))
airlight = airlight_head(image) # or bank / fixed
result = compose_physical_haze(image, density, airlight)
```

9 方法优势与局限

9.1 优势

- **物理可解释**：生成变量明确分为 density 和 atmospheric light，而不是难解释的 RGB residual。
- **颜色更可控**：RGB 颜色主要由 $\mathbf{A}$ 和 clean image 决定，减少扩散模型自由生成颜色导致的偏色。
- **结构更稳定**：depth 作为几何条件影响 density 分布，远近雾感更符合物理直觉。
- **真实域迁移更温和**：Stage2 使用 A-bank 和 CLIP 方向约束，不把真实雾图 prototype 强行套到图像内容上。

9.2 局限

- 全局 $\mathbf{A}$ 假设较简化，复杂光照或局部彩色雾场可能需要空间变化的 $\mathbf{A}(\mathbf{x})$ 。
- density target 由 paired clean/hazy 反解得到，仍依赖大气光估计质量。
- 线性物理 renderer 无法直接生成非均匀散射中的复杂光晕、雨雾颗粒等高频现象。
- CLIP 方向约束是语义级约束，对细粒度物理真实感的约束有限。

10 与工程文件的对应

表 2: 实现文件索引

文件	作用
<code>phys_haze_utils.py</code>	density / A 估计、carrier 转换、物理成雾 renderer
<code>train_phys_hazegen.py</code>	Stage1 与 Stage2 训练入口、损失函数和日志预览
<code>inference_phys_hazegen.py</code>	推理采样与物理合成流程
<code>configs/train/phys_stage.yaml</code>	ControlLDM、ControlNet、VAE、CLIP、diffusion 配置
<code>README_PHYS_HAZE.md</code>	方法分支简要说明

11 总结

PhysHazeDiffusion 的核心思想可以概括为：

Diffusion learns $d(\mathbf{x})$, Airlight controls $\mathbf{A}$, Physics renders $\mathbf{I}$.

相比直接学习 RGB 雾图或 RGB residual，该方法把成雾过程拆成可解释的物理中间变量，使扩散模型专注于雾密度的空间结构，而把颜色、亮度和雾幕效果交给大气光与物理公式。这也是当前实验中效果较稳定的主要原因。